



Raspberry Pi



Raspberry Pi

SYSTEM DESCRIPTION



COURSE OUTCOME

Illustrate the bus fabric and single cycle I/O of
RP2040

Apply

Bus Fabric:

- AHB-Lite Crossbar
- Atomic Register Access
- APB Bridge
- Narrow IO Register Writes



Bus Fabric:

- AHB-Lite Crossbar
- Atomic Register Access
- APB Bridge
- Narrow IO Register Writes



Processor Subsystem:

- SIO (Serial Input/Output)
- Interrupts
- Event Signals
- Debug

You'll learn

Bus Fabric:

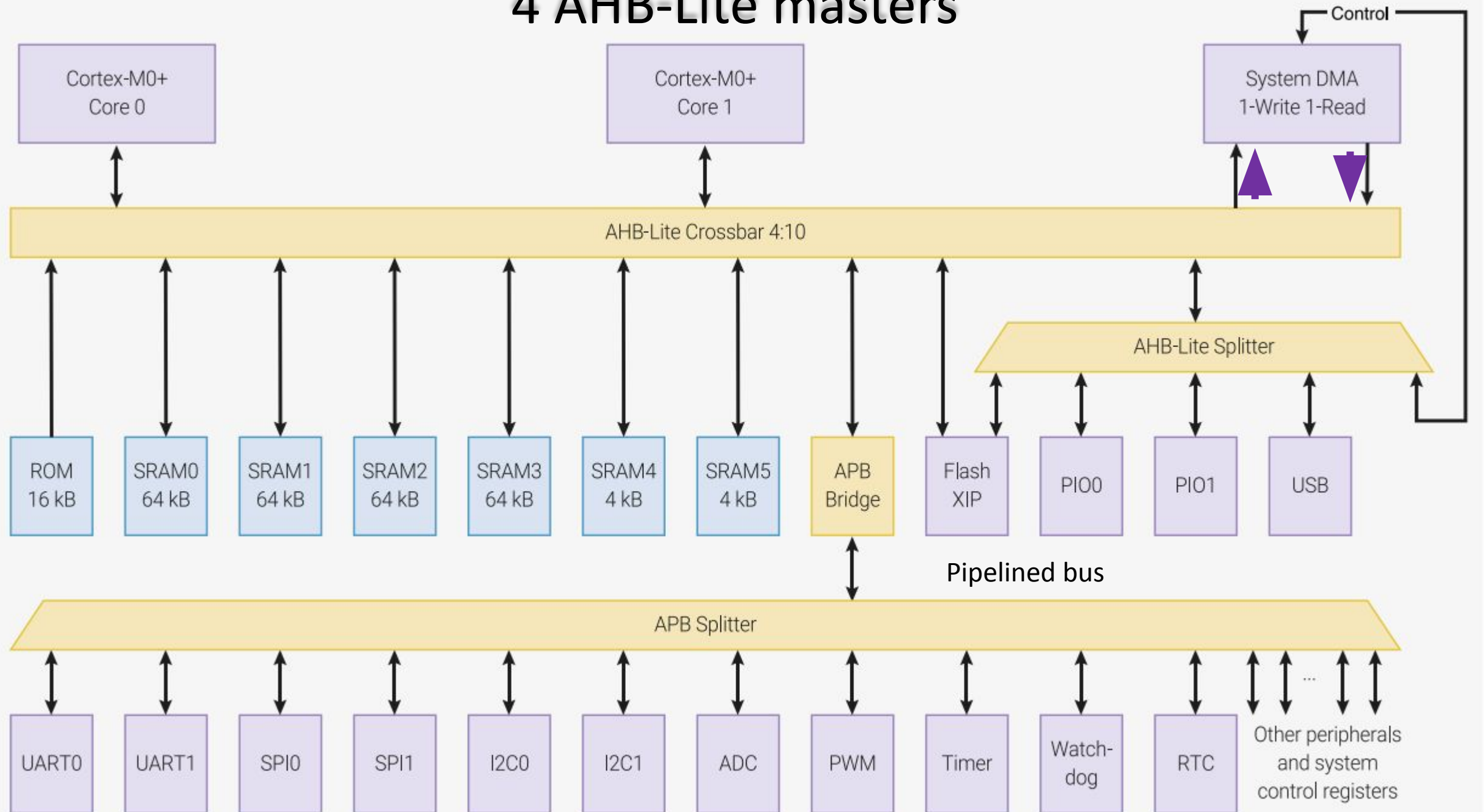
- AHB-Lite Crossbar

Detail the structure of the bus fabric

- Atomic Register Access
- APB Bridge
- Narrow IO Register Writes



4 AHB-Lite masters



op Section – Masters (Initiators)

The title “**4 AHB-Lite masters**” refers to the four components that can initiate data transfers on the AHB-Lite bus:

1. **Cortex-M0+ Core 0**
2. **Cortex-M0+ Core 1**
3. **System DMA** (Direct Memory Access controller) — performs 1 write and 1 read operation independently of the CPU.
4. (Possibly other internal bus masters such as debug or system controller)

AHB-Lite Crossbar (4:10)

- The **crossbar** acts as a **bus interconnect** between **4 masters** and **10 slaves**.
- It allows multiple bus transactions to occur in parallel — improving throughput.
- The label **4:10** means 4 inputs (masters) and 10 outputs (slaves).

AHB-Lite Slaves (Memory and High-Speed Peripherals)

Connected below the AHB-Lite crossbar are memory and high-speed modules:

- **ROM (16 KB)** – Stores firmware or bootloader.
- **SRAM0–SRAM5 (64 KB × 4, 4 KB × 2)** – On-chip data memory blocks.
- **APB Bridge** – Connects slower peripherals via the APB bus.
- **Flash XIP (Execute-in-Place)** – Allows executing code directly from flash memory.
- **PIO0 / PIO1** – General-purpose programmable I/O controllers.
- **USB** – High-speed peripheral device controller.

APB Bridge and Splitter (for Low-Speed Peripherals)

- The **APB (Advanced Peripheral Bus)** is used for **low-speed peripherals** that do not require high bandwidth.
- The **APB Bridge** connects the AHB-Lite high-speed domain to the APB low-speed domain.
- The **APB Splitter** distributes connections to the following peripherals:
 - UART0, UART1 – Serial communication interfaces
 - SPI0, SPI1 – Serial Peripheral Interface modules
 - I2C0, I2C1 – Inter-Integrated Circuit interfaces
 - ADC – Analog-to-Digital Converter
 - PWM – Pulse Width Modulation controller
 - Timer, Watchdog, RTC – Timing and system control peripherals
 - Other control and system registers

System DMA (Direct Memory Access)

- Operates independently from the CPU to transfer data directly between memory and peripherals.
- Reduces CPU load and increases data throughput.

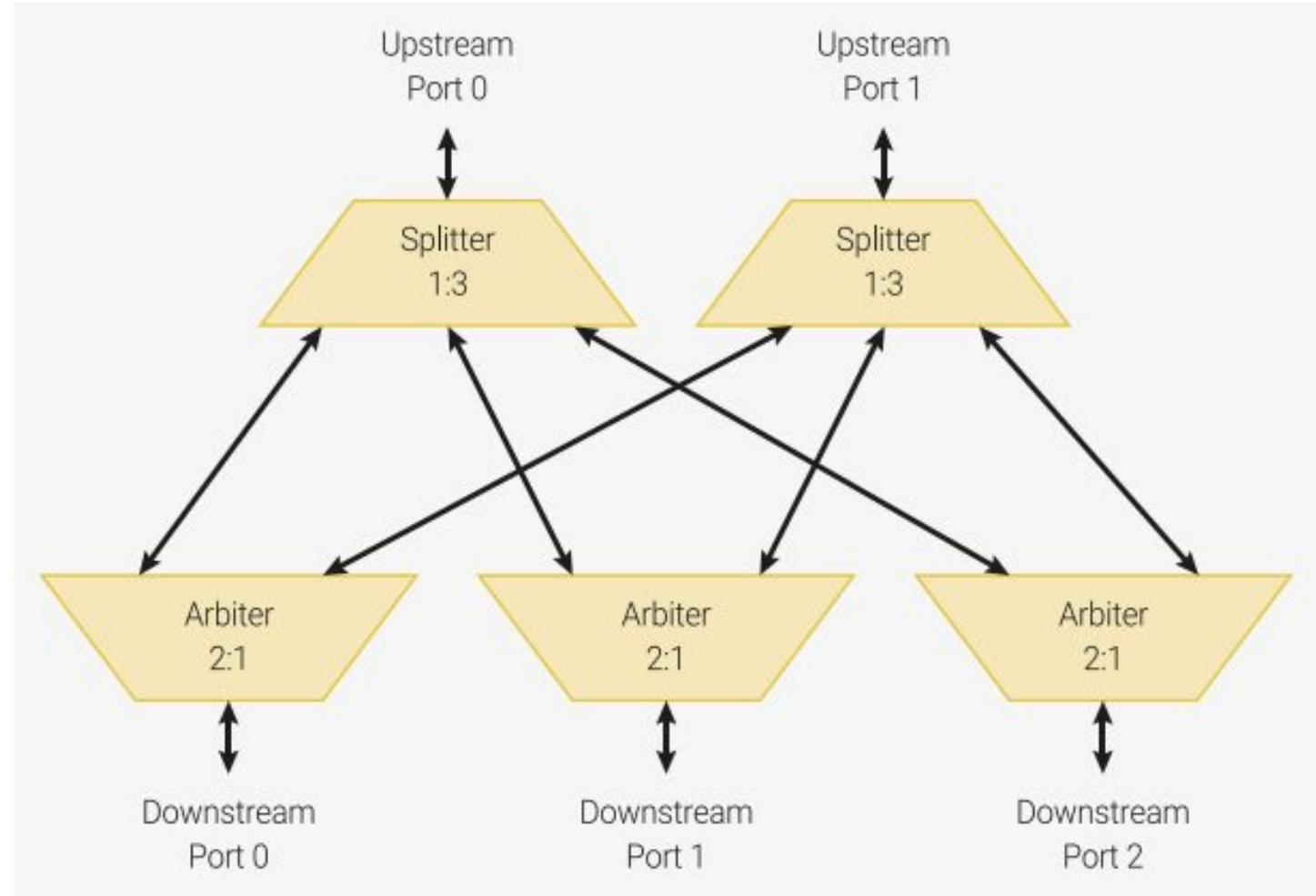
AHB-LITE CROSSBAR

Splitters

- Perform coarse address decode
- Route requests (addresses, write data) to the downstream port indicated by the initial address decode
- Route responses (read data, bus errors) from the correct arbiter back to the upstream port

Arbiters

- Manage concurrent requests to a downstream port
- Route responses (read data, bus errors) to the correct splitter
- Implement bus priority rules



Bus Priority

When there are multiple simultaneous accesses to same arbiter, any requests from *high-priority masters (priority level 1)* will be considered before any requests from *low-priority masters (priority 0)*. If multiple masters of the same priority level attempt to access the same slave simultaneously, a *round-robin tie break* is applied, i.e. the arbiter grants access to each master in turn.

Bus Priority

When accessing a slave with **zero** wait states, such as SRAM (i.e. can be accessed once per system clock cycle), high priority masters will never observe any slowdown. it does however mean a low-priority master may get stalled until there is a free cycle.

Bus Performance Counters

- The performance counters automatically count accesses to the main AHB-Lite crossbar arbiters. This can assist in diagnosing performance issues, in high-traffic use cases.
- 24-bit saturating counter

You'll learn:

AHB-Lite Crossbar

Atomic Register Access

Discern the 4 types of
atomic register access

- APB Bridge
- Narrow IO Register Writes



Atomic Register Access

Each peripheral register block is allocated 4kB of address space, with registers accessed using one of 4 methods, selected by address decode.

- Addr + 0x0000 : normal read write access
- Addr + 0x1000 : atomic XOR on write
- Addr + 0x2000 : atomic bitmask set on write
- Addr + 0x3000 : atomic bitmask clear on write



This allows individual fields of a control register to be modified without performing a read-modify-write sequence in software:

You'll learn:

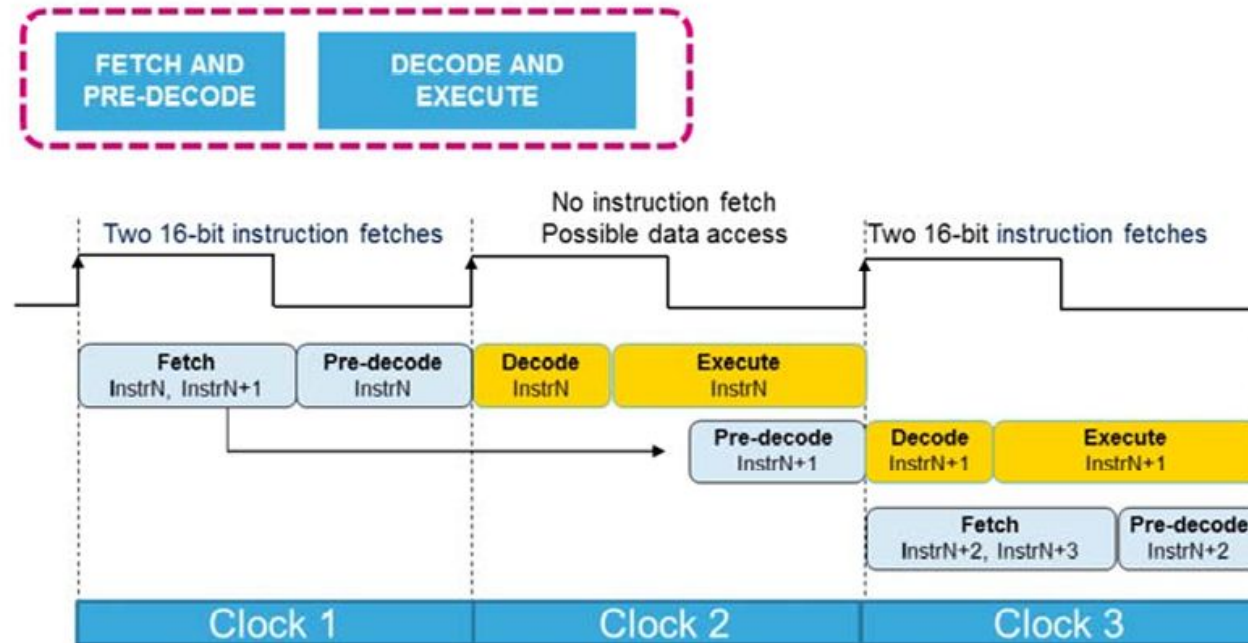
APB Bridge & Narrow IO Register Writes

Summarize the APB bridge and narrow IO
register writes



Instruction Cycles

ARM Cortex-M0+ → 2-stage pipeline



Advanced Performance Bus (**APB** Bridge)

APB bus

The APB bridge interfaces the high-speed main AHB-Lite interconnect to the lower-bandwidth peripherals

|| AHB –Zero / APB – Cycle penalty||

- APB bus accesses take 2 cycles minimum (setup phase and access phase)
- The bridge adds an additional cycle to read accesses, as the bus request and response are registered
- The bridge adds two additional cycles to write accesses, as the APB setup phase can not begin until the AHB-Lite write data is valid

Throughput: APB bus < AHB Lite

Instruction Cycles



Narrow IO Register Writes

Memory-mapped IO registers on RP2040 ignore the width of bus read/write accesses.

They treat all writes as though they were **32 bits** in size.

*To update part of an IO register, without a read-modify-write sequence, the best solution on **RP2040** is **atomic set/clear/XOR***

List of Registers

The Bus Fabric registers start at a base address of 0x40030000 (defined as BUSCTRL_BASE in SDK).

Table 4. List of
BUSCTRL registers

Offset	Name	Info
0x00	BUS_PRIORITY	Set the priority of each master for bus arbitration.

2.1. Bus Fabric

RP2040 Datasheet

Offset	Name	Info
0x04	BUS_PRIORITY_ACK	Bus priority acknowledge
0x08	PERFCTR0	Bus fabric performance counter 0



Processor Subsystem:

Define processor subsystem with a block diagram

- SIO (Serial Input/Output)
- Interrupts
- Event Signals
- Debug



Processor Subsystem:

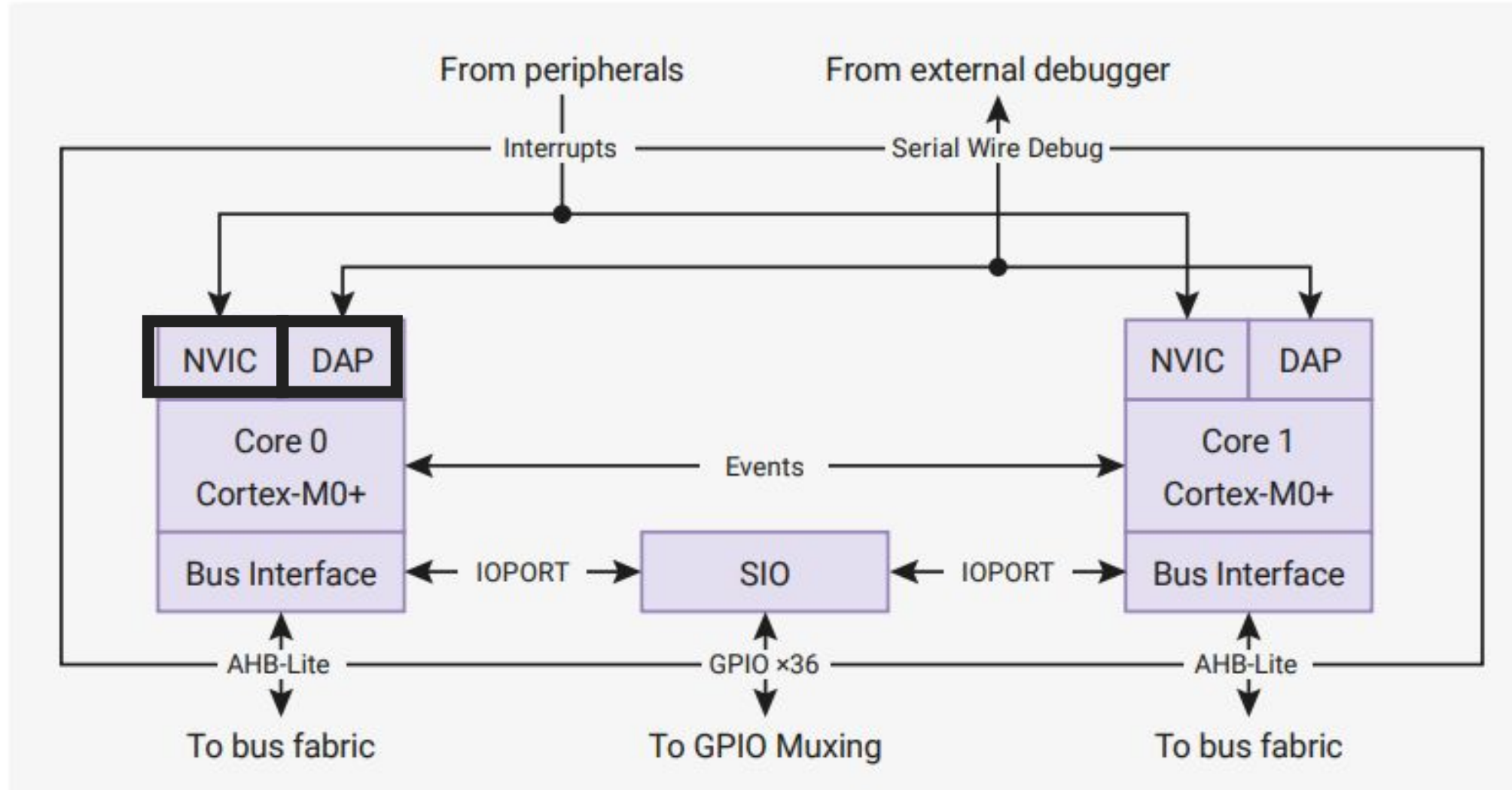
Define processor subsystem with a block diagram

- SIO (Serial Input/Output)
- Interrupts
- Event Signals
- Debug

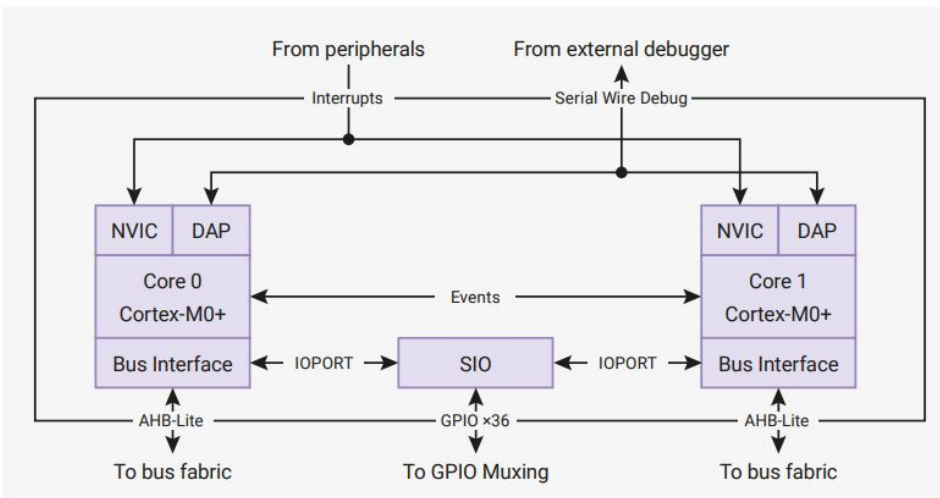
Processor Subsystem

Intended
Learning
Outcome

Define processor subsystem with a block diagram



Processor Subsystem

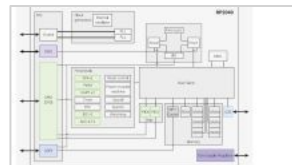
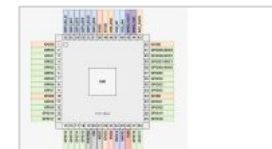
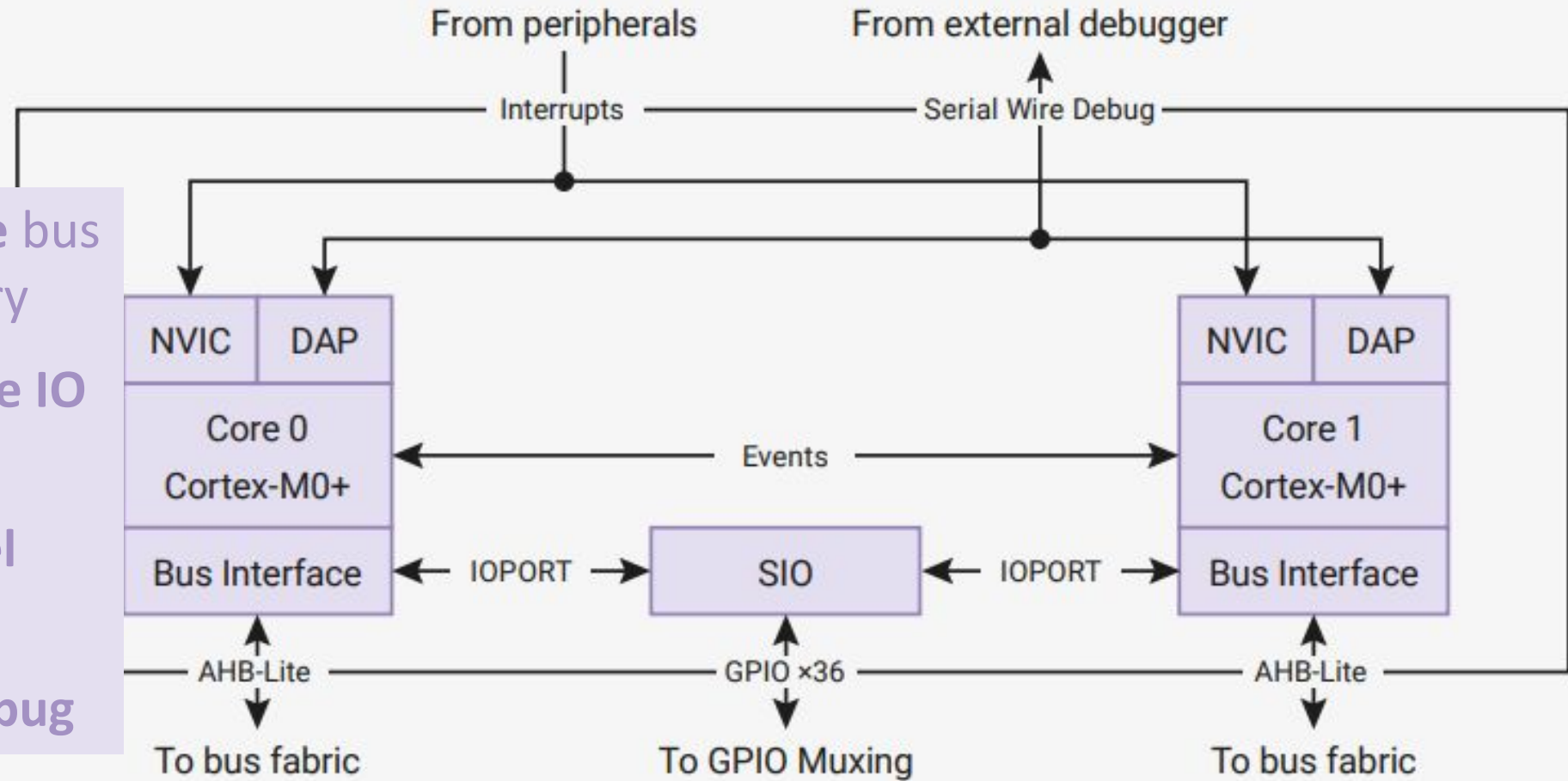


The processors use a number of interfaces to communicate with the rest of the system:

- Each processor uses its own independent 32-bit **AHB-Lite** bus to access memory and memory-mapped peripherals
- The **single-cycle IO** block provides high-speed, deterministic access to GPIOs via each processor's IOPORT
- **26 system-level interrupts** are routed to both processors
- A multi-drop **Serial Wire Debug** bus provides debug access to both processors from an external debug host

Processor Subsystem

- 32-bit AHB-Lite bus to access memory
- The single-cycle IO access to
- 26 system-level interrupts
- Serial Wire Debug



SIO – Single Cycle IO

Intended
Learning
Outcome

Discuss the registers in SIO and find how to choose between different cores and controlling GPIO

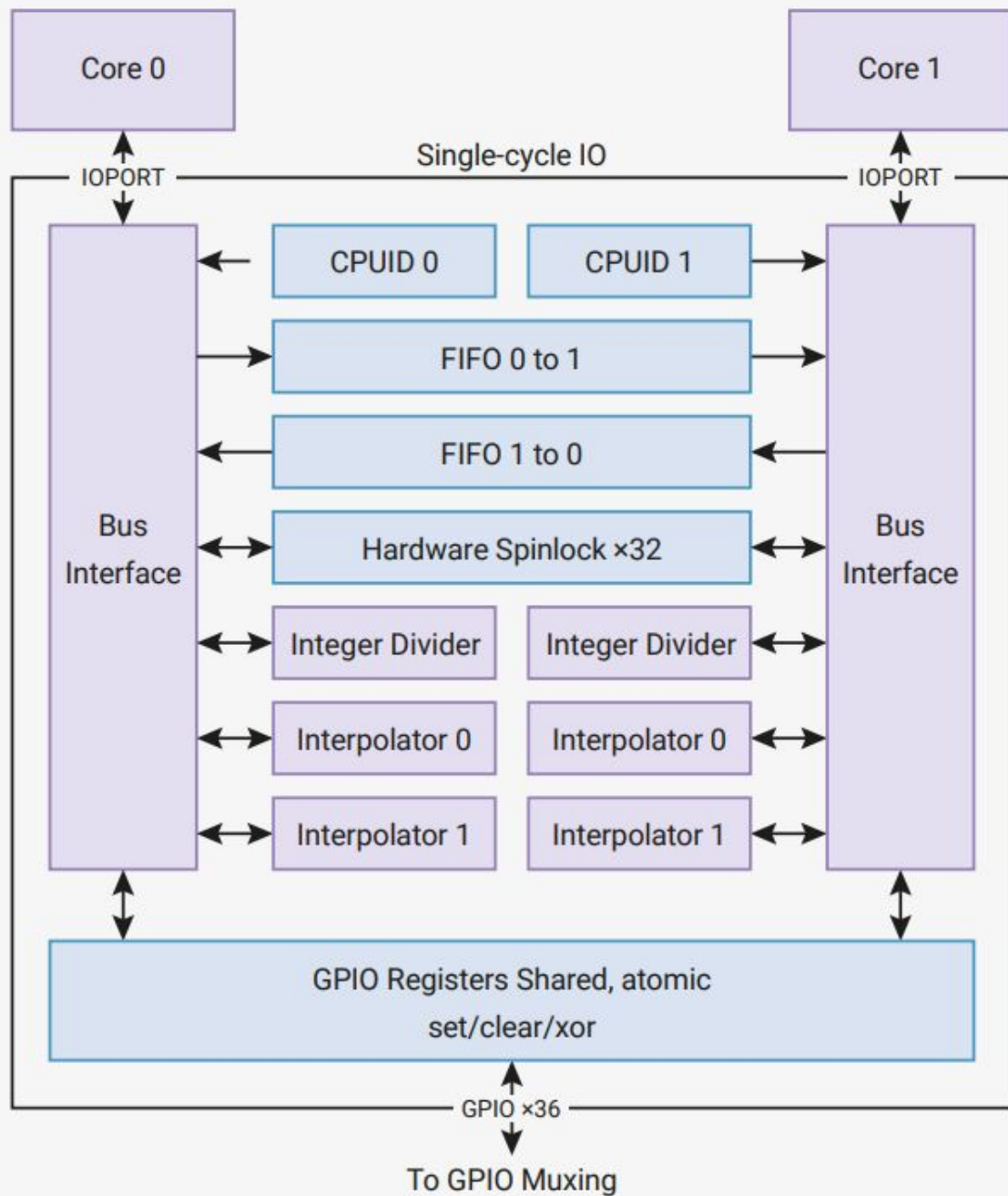
- Low-latency
- Deterministic access from the processors

IOPORT AND OTHER SPECIAL OPERATION



0xd0000000 0xd000017c

- All IOPORT reads and writes (and therefore all SIO accesses) take place in exactly one cycle
- Main AHB-Lite system bus, requires two cycles for a load or store,



1. CPUID
2. GPIO Control
3. Hardware Spinlocks
4. Inter-processor FIFOs (Mailboxes)
5. Integer Divider
6. Interpolator

CPUID

The register CPUID is the first register in the IOPORT space. Core 0 reads a value of 0 when accessing this address, and core 1 reads a value of 1.

GPIO Control

The processors have access to GPIO registers for fast and direct control of pins with GPIO functionality

- Output registers, **GPIO_OUT** and **GPIO_HI_OUT**
- Output enable registers, **GPIO_OE** and **GPIO_HI_OE**, are used to enable the output driver
- Input registers, **GPIO_IN** and **GPIO_HI_IN**, allow the processor to sample the current state of the GPIOs

GPIO_x

Bit 31	Bit 30	Bit 29	Bit 28	Bit 27	Bit 26	Bit 25	...	Bit 3	Bit 2	Bit 1	Bit 0
—	—	GPIO29	GPIO28	GPIO27	GPIO26	GPIO25	...	GPIO3	GPIO2	GPIO1	GPIO0

GPIO_HI_x

Bit 31	Bit 30	...	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
—	—	...	—	—	—	—	GPIO2 9	GPIO2 8	GPIO2 7	GPIO2 6

Hardware Spinlocks

Manage mutually-exclusive access to shared software resources.

Generally the spinlocks should be used to the short critical sections

Read: attempt to claim the lock. || **Write** (any value): release the lock



Inter-processor FIFOs (Mailboxes)

The SIO contains two FIFOs for passing data, messages or ordered events between the two cores

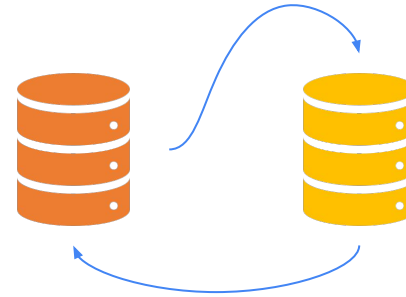
Each FIFO is **32 bits wide**, and **eight entries deep**

One of the FIFOs can only be written by core 0, and read by core 1

FIFO_WR / **FIFO_RD** / **FIFO_ST**
write read

FIFO_ST

- Incoming FIFO contains data (**VLD**)
- Outgoing FIFO has room for more data (**RDY**)
- The incoming FIFO was read from while empty at some point in the past (**ROE**)
- The outgoing FIFO was written to while full at some point in the past (**WOF**)



Core 0 Core 1

The SIO has a FIFO IRQ output for each core, mapped to system IRQ numbers **15** and **16**

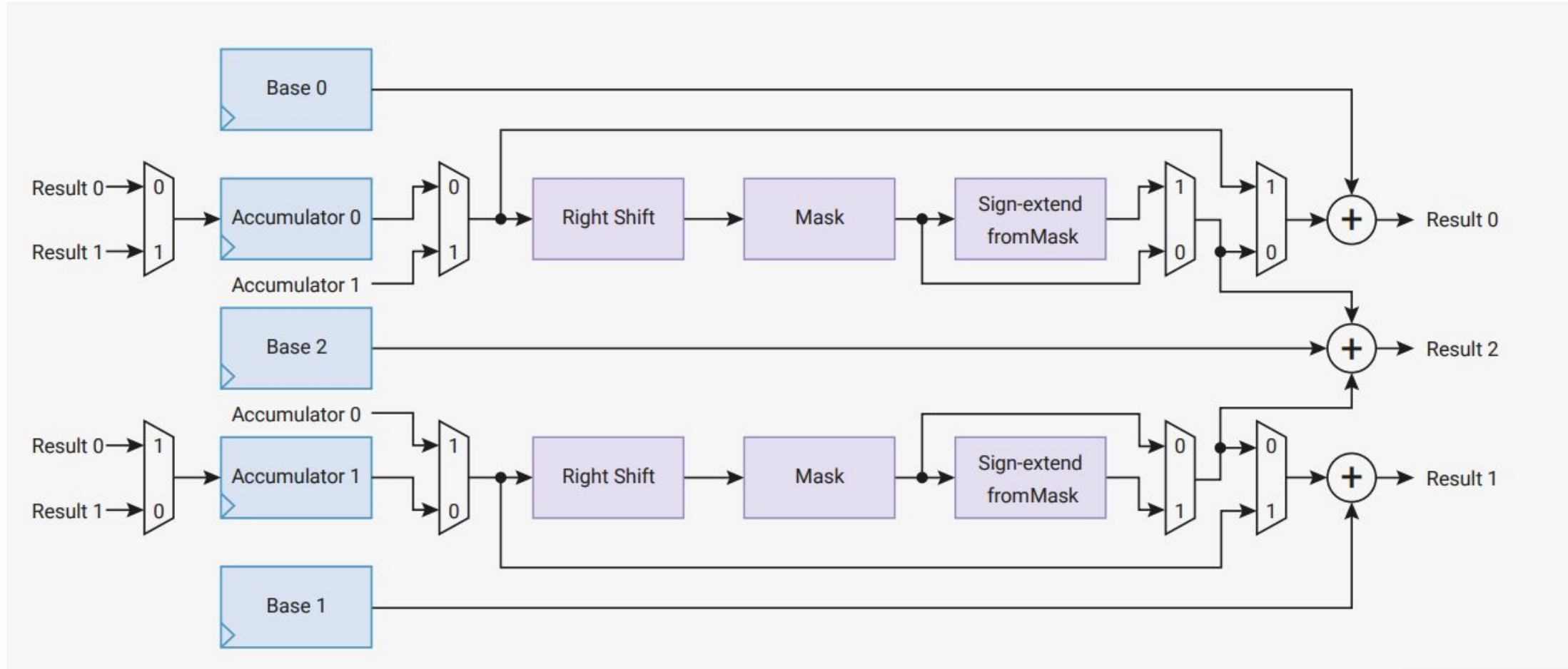
Integer Divider

The SIO provides one 8-cycle signed/unsigned divide/modulo module to each of the cores.

Calculation is started by writing a dividend and divisor to the two argument registers, DIVIDEND and DIVISOR. The divider calculates the quotient $/$ and remainder $\%$ of this division over the next **8** cycles, and on the **9**th cycle the results can be read from the two result registers **DIV_QUOTIENT** and **DIV_REMAINDER**.

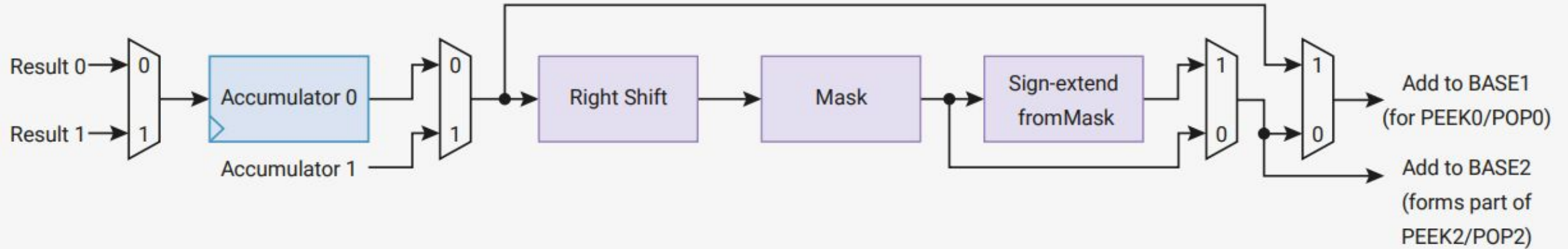
Interpolator

Each core is equipped with two interpolators (INTERP0 and INTERP1) which can accelerate tasks by combining certain preconfigured operations into a single processor cycle.



Examine how interpolator is used in RP2040

Intended Learning
Outcome



Interpolator : Lane operations

Each lane performs these three operations, in sequence:

- A right shift by CTRL_LANE_x_SHIFT (0 to 31 bits)
- A mask of bits from CTRL_LANE_x_MASK_LSB to CTRL_LANE_x_MASK_MSB inclusive (each ranging from bit 0 to bit 31)
- A sign extension from the top of the mask, i.e. take bit CTRL_LANE_x_MASK_MSB and OR it into all more-significant bits, if CTRL_LANE_x_SIGNED is set

For example, if:

- `ACCUM0 = 0xdeadbeef`
- `CTRL_LANE0_SHIFT = 8`
- `CTRL_LANE0_MASK_LSB = 4`
- `CTRL_LANE0_MASK_MSB = 7`
- `CTRL_SIGNED = 1`

Then lane 0 would produce the following results at each stage:

- Right shift by 8 to produce `0x00deadbe`
- Mask bits 7 to 4 to produce `0x00deadbe & 0x000000f0 = 0x000000b0`
- Sign-extend up from bit 7 to produce `0xffffffffb0`

0xdeadbee 11011110101011011011111011101111

f

Original value: 11011110101011011011111011101111

Right shift by 8: 00000000110111101010110110111110

CTRL_LANE0_SHIFT = 8

Mask 00000000 00000000 00000000 11110000

00000000 00000000 11011110 10111110 (0x00deadbe)

& 00000000 00000000 00000000 11110000 (0x000000f0)

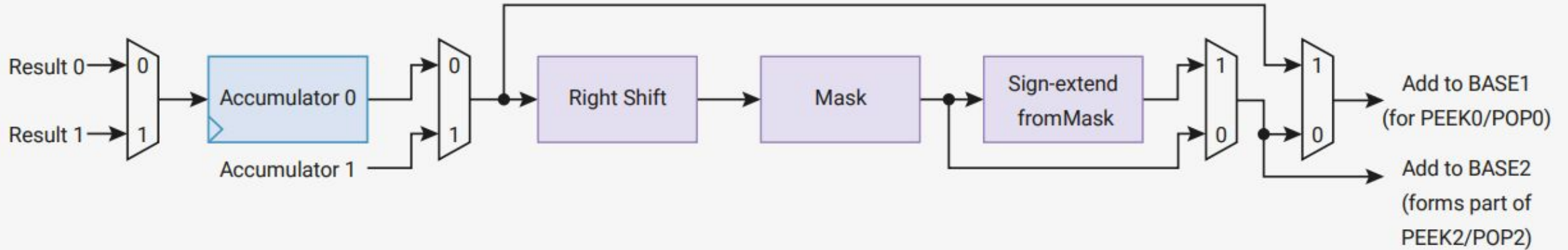
00000000 00000000 00000000 10110000

Mask bits 7 to 4

Original value: 00000000 00000000 00000000 10110000

CTRL_SIGNED = 1

Already positive



Interpolator : Blend Mode

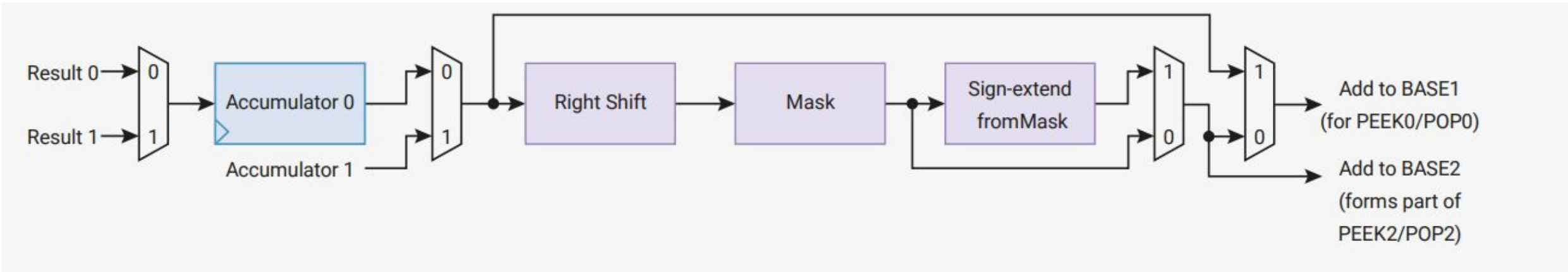
Blend mode is available on **INTERP0** on each core, and is enabled by the CTRL_LANE0_BLEND control flag. It performs linear interpolation,

$$x = x_0 + \alpha(x_1 - x_0), \text{ for } 0 \leq \alpha < 1$$

Blend mode has the following differences from normal mode:

- PEEK0, POP0 return the 8-bit alpha value (the 8 LSBs of the lane 1 shift and mask value), with zeroes in result bits 31 down to 24.
- PEEK1, POP1 return the linear interpolation between BASE0 and BASE1
- PEEK2, POP2 do not include lane 1 result in the addition (i.e. it is BASE2 + lane 0 shift and mask value)

The blend mode seems to perform linear interpolation between two values, which could be useful for smoothly transitioning between values



Interpolator : Clamp Mode

Clamp mode is available on INTERP1 on each core, and is enabled by the CTRL_LANE0_CLAMP control flag. In clamp mode, the PEEK0/POP0 result is the lane value (shifted, masked, sign-extended ACCUM0) clamped between BASE0 and BASE1.

In other words, if the lane value is greater than BASE1, a value of BASE1 is produced; if less than BASE0, a value of BASE0 is produced

This mode appears to control how values are restricted within specific ranges.



Interpolator : Blend Mode

Blend Mode

Smoothly interpolates between BASE0 and BASE1

Uses lower 8 bits of Lane 1 as fraction (α) for mixing

Available only on **Interpolator 0, Lane 0**

Interpolator : Clamp Mode

Clamp Mode

Keeps accumulator value within a defined min–max range

Prevents overflow/underflow when generating indices (safe for arrays)

Works on **any lane (0 or 1)**

- **Nested Interrupts Handling:**
- **Priority Determination**
- **Dynamic Priority Levels**
- **Interrupt Arrangement**
- **NMI Signal Generation**

1.Nested Interrupts Handling: Hardware supports nested interrupts, where a higher-priority interrupt or exception (like HardFault) can pre-empt a lower-priority interrupt. The lower-priority interrupt resumes execution after higher-priority events finish processing. Priority order is established by two factors:

1. Dynamic priority level set per interrupt using **NVIC_IPR 0-7** registers.
2. Lower-numbered IRQs take precedence when multiple interrupts share the same dynamic priority level.

1.Dynamic Priority Level: Cortex-M0+ allocates four priority levels through the two most significant bits of an 8-bit priority field. The lowest **numerically (level 0) is the highest priority**.

2.Interrupt Arrangement: RP2040's interrupt table is designed with a logical default priority order. However, **NVIC_IPR0 through NVIC_IPR7** registers allow individual interrupts' priorities to be adjusted for specific use cases.

3.NMI Signal Generation: The 26 system IRQ signals undergo masking (NMI mask) and logical **ORing** to generate the NMI (Non-Maskable Interrupt) signal for the core. Configuration of NMI masks for each core is possible through **PROC0_NMI_MASK and PROC1_NMI_MASK** in the **Syscfg** register block. When a system interrupt is asserted, and the corresponding NMI mask bit is set, the respective core's NMI is triggered.

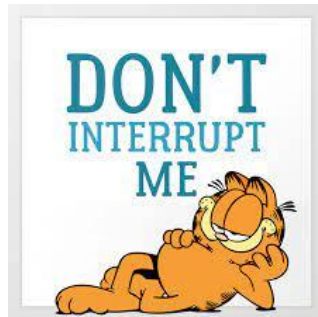
Interrupts – Event Signals –Debug

Intended
Learning
Outcome

Illustrate interrupts and event signals

Interrupts

- Each core is equipped with a standard ARM Nested Vectored Interrupt Controller (NVIC) which has 32 interrupt inputs..
- Each NVIC has the same interrupts routed to it, with the exception of the GPIO interrupts



Interrupts are signals that inform a processor to temporarily pause its current activities and start executing a specific set of instructions related to the interrupt.

Lets Assume

We have a interrupt 0,1 and 2 on core 0

To set the priority of these three interrupts lets us use the NVIC IPR0

MOPLUS: NVIC_IPR0 Register

Offset: 0xe400

Description

Use the Interrupt Priority Registers to assign a priority from 0 to 3 to each of the available interrupts. 0 is the highest priority, and 3 is the lowest.

Note: Writing 1 to an NVIC_ICPR bit does not affect the active state of the corresponding interrupt.

These registers are only word-accessible

Table 96. NVIC_IPR0 Register

Bits	Name	Description	Type	Reset
31:30	IP_3	Priority of interrupt 3	RW	0x0
29:24	Reserved.	-	-	-
23:22	IP_2	Priority of interrupt 2	RW	0x0
21:16	Reserved.	-	-	-
15:14	IP_1	Priority of interrupt 1	RW	0x0
13:8	Reserved.	-	-	-
7:6	IP_0	Priority of interrupt 0	RW	0x0
5:0	Reserved.	-	-	-

00000000 00000000 01000000 10000000

Here, I have given highest priority to 2nd interrupt by giving “00” to bits 23 & 22... second priority to interrupt 1 and the last priority to interrupt 0.

On RP2040, only the lower 26 IRQ signals are connected on the NVIC, and IRQs 26 to 31 are tied to zero (never firing). The core can still be forced to enter the relevant interrupt handler by writing bits 26 to 31 in the **NVIC ISPR** register

Interrupt Set-pending Register

IRQ	Interrupt Source	IRQ	Interrupt Source	IRQ	Interrupt Source	IRQ	Interrupt Source	IRQ	Interrupt Source
0	TIMER_IRQ_0	6	XIP_IRQ	12	DMA_IRQ_1	18	SPI0_IRQ	24	I2C1_IRQ
1	TIMER_IRQ_1	7	PIO0_IRQ_0	13	IO_IRQ_BANK0	19	SPI1_IRQ	25	RTC_IRQ
2	TIMER_IRQ_2	8	PIO0_IRQ_1	14	IO_IRQ_QSPI	20	UART0_IRQ		
3	TIMER_IRQ_3	9	PIO1_IRQ_0	15	SIO_IRQ_PROC0	21	UART1_IRQ		
4	PWM_IRQ_WRAP	10	PIO1_IRQ_1	16	SIO_IRQ_PROC1	22	ADC_IRQ_FIFO		
5	USBCTRL_IRQ	11	DMA_IRQ_0	17	CLOCKS_IRQ	23	I2C0_IRQ		

We have a interrupt 0,1 and 2 on core 0
To set the priority of these two interrupts lets us use the **NVIC_IPRO**

00000000 00000000 01000000 10000000

Here, I have given highest priority to 2nd interrupt by giving “00” to bits 23 &22

However if I want to set interrupt 1 as the non maskable interrupt, then we have to use **PROC0_NMI_Register**

SYSCFG: PROC0_NMI_MASK Register

Offset: 0x00

Description

Processor core 0 NMI source mask

Bits	Description	Type	Reset
31:0	Set a bit high to enable NMI from that IRQ	RW	0x00000000

→ 00000000 00000000 00000000 0000010

A random interrupt masking already in core 0

00000000 10000000 01000100 01000000
00000000 00000000 00000000 00000100 OR
00000000 10000000 01000100 01000100

31	30	29	28	27	26	25	24
----	----	----	----	----	----	----	----

23	22	21	20	19	18	17	16
----	----	----	----	----	----	----	----

15	14	13	12	11	10	9	8
----	----	----	----	----	----	---	---

7	6	5	4	3	2	1	0
---	---	---	---	---	---	---	---

--	--	--	--	--	--	--	--

--	--	--	--	--	--	--	--

--	--	--	--	--	--	--	--

--	--	--	--	--	--	--	--



Event Signals

- The Cortex-M0+ can enter a sleep state until an "event" (or interrupt) takes place, using the **WFE** instruction. Wait For Event
- It can generate events, using the **SEV** instruction. **Send Event**
- On RP2040 the event signals are cross-wired between the two processors, so that an event sent by one processor will be received on the other.

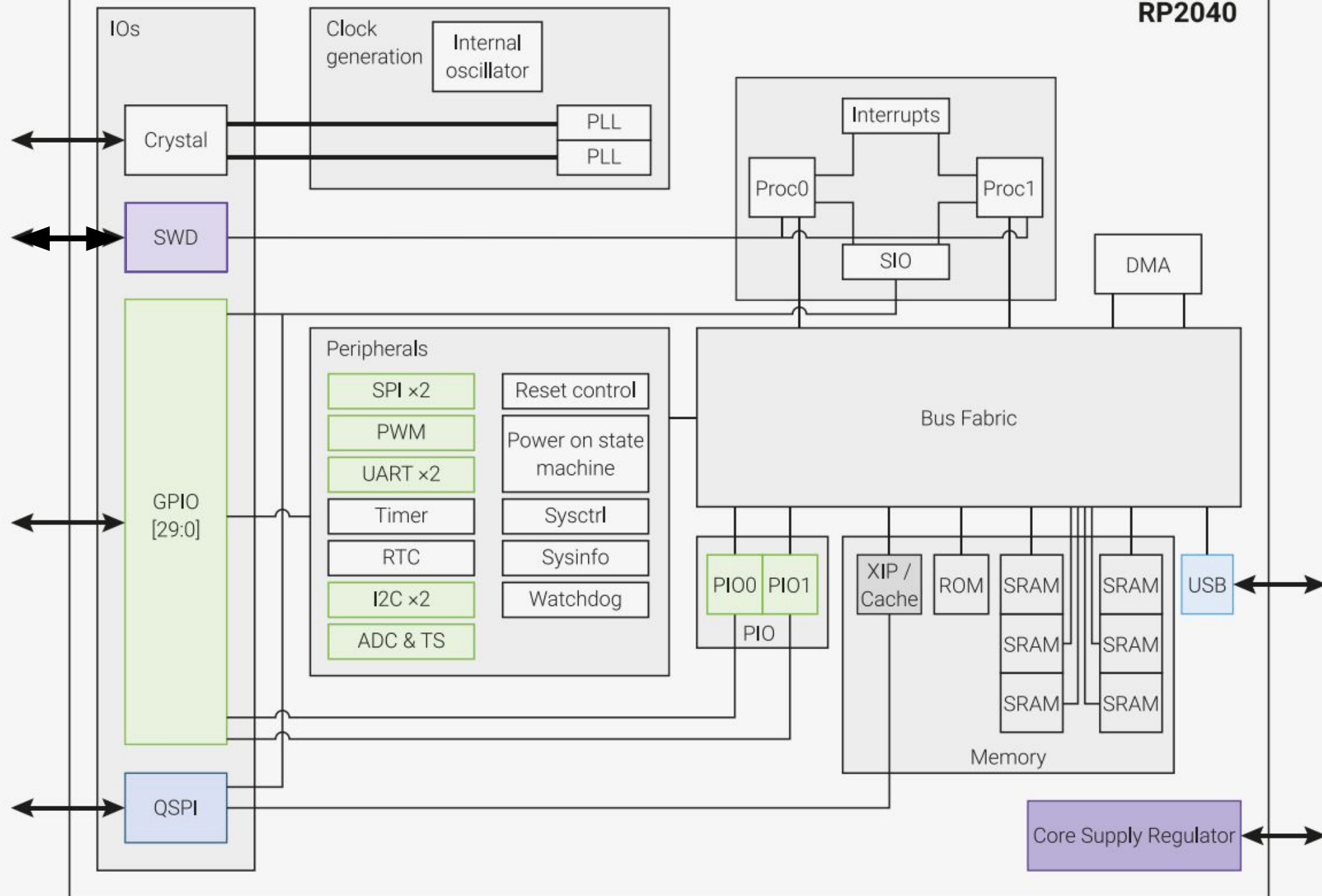
While in a WFE (or WFI) sleep state, the processor can shut off its internal clock gates, consuming much less power.

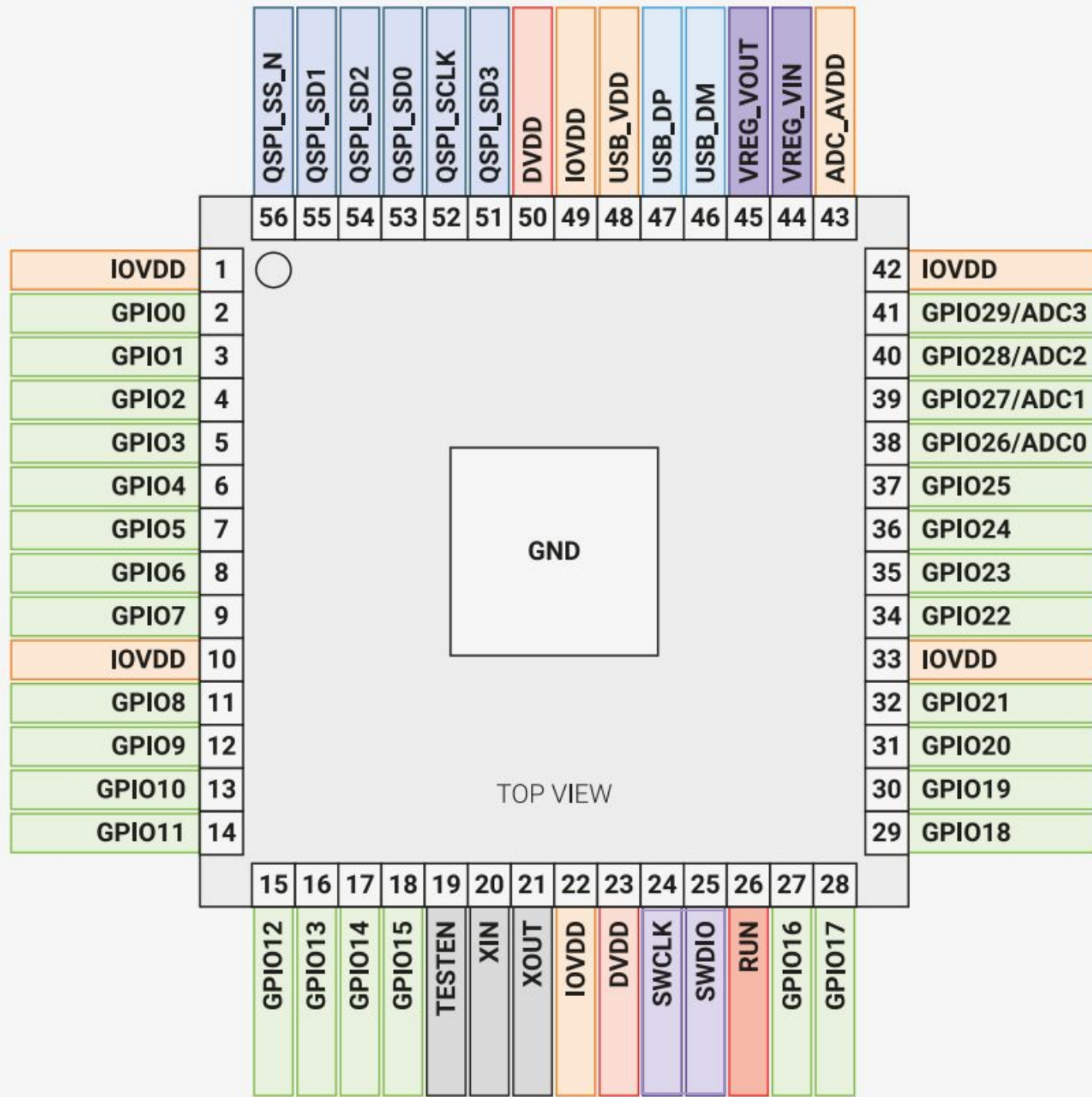


0.2 milliwatts

When both processors are sleeping, and the DMA is inactive, RP2040 as a whole can enter a sleep state, disabling clocks on unused infrastructure such as the busfabric, and waking automatically when one of the processors wakes.

RP2040

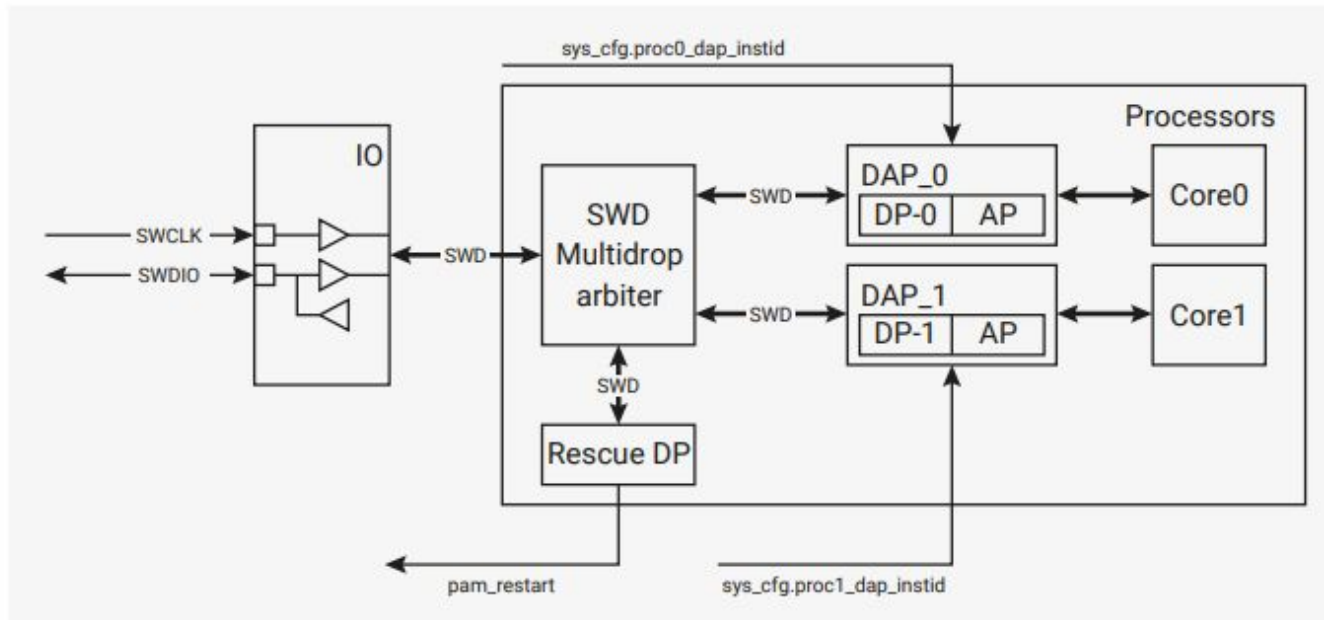




Debug

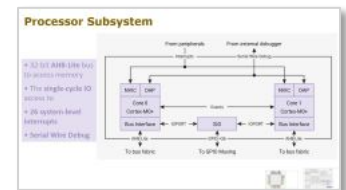
The **2-wire Serial Wire Debug (SWD)** port provides access to hardware and software debug features including:

- Loading firmware into SRAM or external flash memory
- Control of processor execution: run/halt, step, set breakpoints, other standard Arm debug functionality
- Access to processor architectural state
- Access to memory and memory-mapped IO via the system bus



Each DAP will only respond to debug commands if correctly addressed by a SWD TARGETSEL command

Additionally, a Rescue DP is available which is connected to system control features



• Core 0: 0x01002927 • Core 1: 0x11002927 • Rescue DP: 0xf1002927

Rescue DP

The Rescue DP (debug port) is available over the SWD bus and is only intended for use in the specific case where the chip has locked up, for example if code has been programmed into flash which permanently halts the system clock

CDBGPWRUPREQ	Core Debug GPIO Peripheral Write Request.
CDBGPWRUPACK	Core Debug Power Up Acknowledge)

Software control of SWD pins

The SWD pins for Core 0 and Core 1 can be bit-banded via registers in syscfg

SYSCFG: DBGFORCE Register

Offset: 0x14

Description

Directly control the SWD debug port of either processor

Bits	Name	Description	Type	Reset
31:8	Reserved.	-	-	-
7	PROC1_ATTACH	Attach processor 1 debug port to syscfg controls, and disconnect it from external SWD pads.	RW	0x0

Bits	Name	Description	Type	Reset
6	PROC1_SWCLK	Directly drive processor 1 SWCLK, if PROC1_ATTACH is set	RW	0x1
5	PROC1_SWDI	Directly drive processor 1 SWDIO input, if PROC1_ATTACH is set	RW	0x1
4	PROC1_SWDO	Observe the value of processor 1 SWDIO output.	RO	-
3	PROC0_ATTACH	Attach processor 0 debug port to syscfg controls, and disconnect it from external SWD pads.	RW	0x0
2	PROC0_SWCLK	Directly drive processor 0 SWCLK, if PROC0_ATTACH is set	RW	0x1
1	PROC0_SWDI	Directly drive processor 0 SWDIO input, if PROC0_ATTACH is set	RW	0x1
0	PROC0_SWDO	Observe the value of processor 0 SWDIO output.	RO	-